MERN STACK

16 Practical Questions

Complete Setup, Code & Run Guide

Windows • PowerShell • VS Code • React • Node.js • Express • MongoDB

Based on the uploaded MERN question bank and practical journal. Each question includes file creation, code, terminal commands and testing steps.

Before you start

- Install Node.js LTS. Verify in PowerShell: node -v and npm -v. Both should show version numbers.
- Install MongoDB Community Server for the MongoDB practicals. MongoDB Compass is optional; mongosh is used for shell commands.
- Open VS Code → File → Open Folder → Downloads\MERN_Practicals. Use the VS Code terminal (Terminal → New Terminal).
- Create a file: Explorer → right-click the correct folder → New File. Paste the code and press Ctrl+S.
- Run a JavaScript file with node filename.js. Run a Vite React app with npm run dev. Keep the server terminal open.
- Stop a running server with Ctrl+C. Do not run two servers on the same port at the same time.

Create the main working folder

```
cd $HOME\Downloads
mkdir MERN_Practicals
cd MERN_Practicals
```

If MERN_Practicals already exists, do not run mkdir again. Just cd into it.

Project map

```
MERN_Practicals\
  operators.js
  evenNumbers.js
  ExpressDemo\
    app.js
    routes.js
    jsonResponse.js
  SumDemo\
    src\App.jsx
    src\App.css
    src\components\...
```

Practical 1: JavaScript operators and control statements

Question bank: 1 | Journal reference: Journal Practical 1(e), direct match.

Purpose

Shows arithmetic, comparison and if/else control flow.

Create files

1. cd $HOME\Downloads\MERN_Practicals
2. Create operators.js in MERN_Practicals.

Code — operators.js

```
let x = 15;
let y = 10;

console.log("Addition:", x + y);
console.log("Greater Than:", x > y);

if (x > y) {
    console.log("x is greater than y");
} else {
    console.log("y is greater than x");
}
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals
2. node operators.js

Important

> Expected: Addition: 25, Greater Than: true, x is greater than y.

Practical 2: Display a message in the browser using a server response

Question bank: 2 | Journal reference: Journal Practical 6(a), Express home route.

Purpose

Creates an Express server and sends plain text to the browser.

Create files

1. cd $HOME\Downloads\MERN_Practicals
2. mkdir ExpressDemo
3. cd ExpressDemo
4. npm init -y
5. npm install express
6. Create app.js inside ExpressDemo.

Code — app.js

```
const express = require("express");
const app = express();
const PORT = 3000;

app.get("/", (req, res) => {
    res.send("Welcome to Node.js and Express!");
});

app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals\ExpressDemo
2. node app.js
3. Open http://localhost:3000.

Important

> Keep the terminal open. Stop the server with Ctrl+C.

Practical 3: Print even numbers from 1 to 50

Question bank: 3 | Journal reference: Journal loop concept; separate question-bank program.

Purpose

Uses a for loop and modulus operator to print only even numbers.

Create files

1. cd $HOME\Downloads\MERN_Practicals
2. Create evenNumbers.js.

Code — evenNumbers.js

```
for (let i = 1; i <= 50; i++) {
    if (i % 2 === 0) {
        console.log(i);
    }
}
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals
2. node evenNumbers.js

Important

Output is 2, 4, 6, …, 48, 50.

Practical 4: Create different routes in Express

Question bank: 4 | Journal reference: Journal Practical 6(a), Parts G/H, direct match.

Purpose

Creates home, about, contact and students GET routes.

Create files

1. cd $HOME\Downloads\MERN_Practicals\ExpressDemo
2. Create routes.js. Express is already installed in this folder.

Code — routes.js

```
const express = require("express");
const app = express();
const PORT = 3000;

app.get("/", (req, res) => {
    res.send("Welcome to Express Server!");
});
app.get("/about", (req, res) => {
    res.send("This is the About Page.");
});
app.get("/contact", (req, res) => {
    res.send("Contact Us at contact@example.com");
});
app.get("/students", (req, res) => {
    res.send("Student Information Page");
});

app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals\ExpressDemo
2. node routes.js
3. Test http://localhost:3000/ , /about , /contact and /students.

Important

> Stop app.js first if it is using port 3000.

Practical 5: Calculate the sum of two numbers in React

Question bank: 5 | Journal reference: Journal Practical 3(c), direct match.

Purpose

Declares two variables, defines add(), and displays the returned sum with JSX.

Create files

1. Create once: cd $HOME\Downloads\MERN_Practicals
2. npm create vite@latest SumDemo -- --template react
3. Choose ESLint if prompted; allow installation if prompted.
4. cd SumDemo
5. Replace src/App.jsx with the code.

Code — src/App.jsx

```
function App() {
    let num1 = 15;
    let num2 = 25;

    function add(a, b) {
        return a + b;
    }

    return (
        <>
            <h1>Practical 3C</h1>
            <h2>Name: Ayesha Sarguru</h2>
            <h2>First Number: {num1}</h2>
            <h2>Second Number: {num2}</h2>
            <h2>Sum = {add(num1, num2)}</h2>
        </>
    );
}
export default App;
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals\SumDemo
2. npm run dev
3. Open the Vite URL printed in terminal (usually http://localhost:5173).

Practical 6: Send a JSON response from Express

Question bank: 6 | Journal reference: Journal JSON-response pattern; question-bank endpoint.

Purpose

Uses res.json() to return a JavaScript object as JSON.

Create files

1. cd $HOME\Downloads\MERN_Practicals\ExpressDemo
2. Create jsonResponse.js.

Code — jsonResponse.js

```
const express = require("express");
const app = express();
const PORT = 3000;

app.get("/", (req, res) => {
    res.json({
        message: "Welcome to MERN Stack",
        status: "Success"
    });
});

app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals\ExpressDemo
2. node jsonResponse.js
3. Open http://localhost:3000 and check message and status.

Important

> Stop any other server using port 3000 before running this.

Practical 7: Apply CSS styling to a React component

Question bank: 7 | Journal reference: Journal Practical 3(d); CSS declarations retained.

Purpose

Styles a React component with centered blue text, light-yellow background, padding, rounded corners and Arial font.

Create files

1. Open SumDemo/src/App.css and replace its contents.
2. Open SumDemo/src/App.jsx and replace its contents.

Code — src/App.css

```
.container {
    text-align: center;
    color: blue;
    background-color: lightyellow;
    padding: 20px;
    border-radius: 10px;
    font-family: Arial;
}
```

Code — src/App.jsx

```
import "./App.css";

function App() {
    return (
        <div className="container">
            <h1>Practical 3D</h1>
            <h2>Ayesha Sarguru</h2>
            <h1>Welcome to React</h1>
            <p>Learning CSS in React</p>
        </div>
    );
}
export default App;
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals\SumDemo
2. npm run dev
3. Open or refresh the Vite URL.

Important

> The journal extract omits the opening div in JSX. This valid version adds div className="container" so the CSS class is applied.

Practical 8: Create a MongoDB database, collection and insert records

Question bank: 8 | Journal reference: Journal Practical 7, MongoDB basics.

Purpose

Creates CollegeDB, a students collection and inserts three documents.

Create files

1. Ensure MongoDB Community Server is running.
2. Open PowerShell and run mongosh.
3. Enter the following commands at the mongosh prompt.

Code — mongosh commands

```
use CollegeDB

db.createCollection("students")

db.students.insertMany([
    { name: "Ayesha", course: "BSc IT", marks: 90 },
    { name: "Rahul", course: "BSc IT", marks: 85 },
    { name: "Neha", course: "BSc IT", marks: 92 }
])

show collections
db.students.find().pretty()
```

Run and verify

1. Run the commands inside mongosh, not normal PowerShell.

Important

> MongoDB can create the database and collection on first insert; createCollection is included because the question explicitly asks for it.

Practical 9: Create multiple React components and render them on one page

Question bank: 9 | Journal reference: Journal Practical 4(a), direct match.

Purpose

Creates Header and Footer components and imports them into App.jsx.

Create files

1. Inside SumDemo/src create folder components.
2. Create Header.jsx and Footer.jsx inside components.
3. Replace SumDemo/src/App.jsx.

Code — src/components/Header.jsx

```
function Header() {
    return <h1>Welcome to MERN Stack Development</h1>;
}
export default Header;
```

Code — src/components/Footer.jsx

```
function Footer() {
    return <h3>Thank You for Visiting</h3>;
}
export default Footer;
```

Code — src/App.jsx

```
import Header from "./components/Header";
import Footer from "./components/Footer";

function App() {
    return (
        <>
            <h1>Practical 4A</h1>
            <h2>Ayesha Sarguru</h2>
            <Header />
            <p>This is the main content of the application.</p>
            <Footer />
        </>
    );
}
export default App;
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals\SumDemo
2. npm run dev
3. Open the Vite URL and verify both components appear.

Practical 10: Fetch data from an API in React

Question bank: 10 | Journal reference: Journal Practical 8, API creation and integration; compact runnable fetch demonstration.

Purpose

React fetches student data from an Express endpoint and renders the returned array.

Create files

1. cd $HOME\Downloads\MERN_Practicals
2. mkdir FetchAPI
3. cd FetchAPI
4. npm init -y
5. npm install express cors
6. Create server.js.
7. Replace SumDemo/src/App.jsx with the React code.

Code — FetchAPI/server.js

```
const express = require("express");
const cors = require("cors");
const app = express();
app.use(cors());

const students = [
    { id: 1, name: "Ayesha", course: "BSc IT" },
    { id: 2, name: "Rahul", course: "BSc IT" },
    { id: 3, name: "Neha", course: "BSc IT" }
];

app.get("/api/students", (req, res) => {
    res.json(students);
});

app.listen(5000, () => {
    console.log("API running at http://localhost:5000");
});
```

Code — SumDemo/src/App.jsx

```
import { useEffect, useState } from "react";

function App() {
    const [students, setStudents] = useState([]);

    useEffect(() => {
        fetch("http://localhost:5000/api/students")
            .then(response => response.json())
            .then(data => setStudents(data))
            .catch(error => console.error("Error:", error));
    }, []);

    return (
        <div>
            <h1>Students from API</h1>
            {students.map(student => (
                <p key={student.id}>
```

```
                    {student.name} - {student.course}
                </p>
            ))}
        </div>
    );
}
export default App;
```

Run and verify

1. Terminal 1: cd $HOME\Downloads\MERN_Practicals\FetchAPI
2. node server.js
3. Terminal 2: cd $HOME\Downloads\MERN_Practicals\SumDemo
4. npm run dev
5. Open the Vite URL. Keep both terminals running.

Important

> This is a simple local API demo. The journal version also demonstrates storing and retrieving data with MongoDB.

Practical 11: Pass data between React components using props

Question bank: 11 | Journal reference: Journal Practical 4, props topic.

Purpose

Passes name and course from parent App to child Student using props.

Create files

1. Create SumDemo/src/components/Student.jsx.
2. Replace SumDemo/src/App.jsx.

Code — src/components/Student.jsx

```
function Student(props) {
    return (
        <>
            <h2>Student Name: {props.name}</h2>
            <h2>Course: {props.course}</h2>
        </>
    );
}
export default Student;
```

Code — src/App.jsx

```
import Student from "./components/Student";

function App() {
    return (
        <>
            <h1>Practical 11 - Props</h1>
            <Student name="Ayesha Sarguru" course="BSc IT" />
        </>
    );
}
export default App;
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals\SumDemo
2. npm run dev
3. Open the Vite URL and verify the child component displays both values.

Practical 12: Create an API to add user details using Express and MongoDB

Question bank: 12 | Journal reference: Journal Practical 8/9, add-user endpoint.

Purpose

Accepts user details through a POST request and inserts them into UserDB.users.

Create files

1. Ensure MongoDB is running.
2. cd $HOME\Downloads\MERN_Practicals
3. mkdir AddUserAPI
4. cd AddUserAPI
5. npm init -y
6. npm install express mongodb cors
7. Create server.js.

Code — AddUserAPI/server.js

```
const express = require("express");
const cors = require("cors");
const { MongoClient } = require("mongodb");

const app = express();
const PORT = 5000;
const client = new MongoClient("mongodb://127.0.0.1:27017");
let usersCollection;

app.use(cors());
app.use(express.json());

async function connectDB() {
    try {
        await client.connect();
        const db = client.db("UserDB");
        usersCollection = db.collection("users");
        console.log("MongoDB connected successfully");
    } catch (error) {
        console.error("MongoDB connection error:", error);
    }
}
connectDB();

app.post("/api/users", async (req, res) => {
    try {
        const user = {
            name: req.body.name,
            email: req.body.email,
            phone: req.body.phone,
            city: req.body.city
        };
        const result = await usersCollection.insertOne(user);
        res.status(201).json({
            message: "User added successfully",
            id: result.insertedId
```

```
        });
    } catch (error) {
        res.status(500).json({ message: "Error adding user" });
    }
});

app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Code — Postman raw JSON body

```
{
    "name": "Priya",
    "email": "priya@example.com",
    "phone": "9876543211",
    "city": "Mumbai"
}
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals\AddUserAPI
2. node server.js
3. In Postman select POST and URL http://localhost:5000/api/users.
4. Choose Body → raw → JSON, paste the sample body and click Send.
5. Verify in mongosh: use UserDB then db.users.find().pretty().

Important

> MongoDB must be running before the API can save data.

Practical 13: Build a To-do list interface using React

Question bank: 13 | Journal reference: No exact To-do practical in the journal; question-bank implementation.

Purpose

Uses useState to add tasks, mark tasks complete and delete tasks.

Create files

1. Replace SumDemo/src/App.jsx with this code.

Code — SumDemo/src/App.jsx

```
import { useState } from "react";

function App() {
    const [task, setTask] = useState("");
    const [tasks, setTasks] = useState([]);

    function addTask(event) {
        event.preventDefault();
        if (task.trim() === "") return;
        setTasks([...tasks, {
            id: Date.now(), text: task, completed: false
        }]);
        setTask("");
    }

    function toggleTask(id) {
        setTasks(tasks.map(item =>
            item.id === id
                ? { ...item, completed: !item.completed }
                : item
        ));
    }

    function deleteTask(id) {
        setTasks(tasks.filter(item => item.id !== id));
    }

    return (
        <div>
            <h1>Practical 13 - To-do List</h1>
            <form onSubmit={addTask}>
                <input value={task}
                    onChange={e => setTask(e.target.value)}
                    placeholder="Enter a task" />
                <button type="submit">Add Task</button>
            </form>
            <ul>
                {tasks.map(item => (
                    <li key={item.id}>
                        <span onClick={() => toggleTask(item.id)}
                            style={{ textDecoration: item.completed
                                ? "line-through" : "none" }}>
                            {item.text}
                        </span>
                        {" "}
                        <button onClick={() => deleteTask(item.id)}>
                            Delete
```

```
                        </button>
                    </li>
                ))}
            </ul>
        </div>
    );
}
export default App;
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals\SumDemo
2. npm run dev
3. Open Vite URL. Add a task, click its text to mark complete, or click Delete.

Important

> Tasks are stored in React state and reset when the page is refreshed.

Practical 14: Create different routes in Express

Question bank: 14 | Journal reference: Journal Practical 6(a), Parts G/H; duplicate routes question in the question bank.

Purpose

Repeats the Express routes task. A separate file and port avoid overwriting or port conflicts.

Create files

1. Create routes_q14.js inside ExpressDemo.

Code — ExpressDemo/routes_q14.js

```
const express = require("express");
const app = express();
const PORT = 3001;

app.get("/", (req, res) => {
    res.send("Welcome to Express Server!");
});
app.get("/about", (req, res) => {
    res.send("This is the About Page.");
});
app.get("/contact", (req, res) => {
    res.send("Contact Us at contact@example.com");
});
app.get("/students", (req, res) => {
    res.send("Student Information Page");
});

app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals\ExpressDemo
2. node routes_q14.js
3. Test http://localhost:3001/ , /about , /contact and /students.

Important

> Port 3001 is used so this practical can run alongside another Express exercise.

Practical 15: React routing — multiple pages and navigation

Question bank: 15 | Journal reference: Journal Practical 5(b)/(c), React routing.

Purpose

Uses react-router-dom to navigate among Home, About and Contact without a full page reload.

Create files

1. cd $HOME\Downloads\MERN_Practicals\SumDemo
2. npm install react-router-dom
3. Create src/pages folder.
4. Create Home.jsx, About.jsx and Contact.jsx inside src/pages.
5. Replace src/App.jsx.

Code — src/pages/Home.jsx

```
function Home() { return <h2>Home Page</h2>; }
export default Home;
```

Code — src/pages/About.jsx

```
function About() { return <h2>About Page</h2>; }
export default About;
```

Code — src/pages/Contact.jsx

```
function Contact() { return <h2>Contact Page</h2>; }
export default Contact;
```

Code — SumDemo/src/App.jsx

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";
import Contact from "./pages/Contact";

function App() {
    return (
        <BrowserRouter>
            <h1>Practical 15 - React Routing</h1>
            <nav>
                <Link to="/">Home</Link>{" | "}
                <Link to="/about">About</Link>{" | "}
                <Link to="/contact">Contact</Link>
            </nav>
            <Routes>
                <Route path="/" element={<Home />} />
                <Route path="/about" element={<About />} />
                <Route path="/contact" element={<Contact />} />
            </Routes>
        </BrowserRouter>
    );
}
export default App;
```

Run and verify

1. cd $HOME\Downloads\MERN_Practicals\SumDemo
2. npm run dev

3. Open Vite URL and click Home, About and Contact.

Practical 16: Integrate React frontend with Node and Express backend

Question bank: 16 | Journal reference: Journal Practical 10, Full Stack Execution and Validation.

Purpose

Starts a React client and Express API separately. React fetches and adds records through HTTP requests.

Create files

1. cd $HOME\Downloads\MERN_Practicals
2. mkdir FullStackDemo
3. cd FullStackDemo
4. npm init -y
5. npm install express cors
6. Create server.js.
7. Replace SumDemo/src/App.jsx with the frontend code.

Code — FullStackDemo/server.js

```
const express = require("express");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());

let users = [
    { id: 1, name: "Ayesha", course: "BSc IT" },
    { id: 2, name: "Rahul", course: "BSc IT" }
];

app.get("/api/users", (req, res) => res.json(users));

app.post("/api/users", (req, res) => {
    const user = {
        id: Date.now(),
        name: req.body.name,
        course: req.body.course
    };
    users.push(user);
    res.status(201).json({
        message: "User added successfully", user
    });
});

app.listen(5000, () => {
    console.log("Backend running at http://localhost:5000");
});
```

Code — SumDemo/src/App.jsx

```
import { useEffect, useState } from "react";

function App() {
    const [users, setUsers] = useState([]);
    const [name, setName] = useState("");
    const [course, setCourse] = useState("");
```

```
    async function loadUsers() {
        const response = await fetch("http://localhost:5000/api/users");
        setUsers(await response.json());
    }

    useEffect(() => { loadUsers(); }, []);

    async function addUser(event) {
        event.preventDefault();
        await fetch("http://localhost:5000/api/users", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ name, course })
        });
        setName("");
        setCourse("");
        loadUsers();
    }

    return (
        <div>
            <h1>Practical 16 - Full Stack Integration</h1>
            <form onSubmit={addUser}>
                <input placeholder="Name" value={name}
                    onChange={e => setName(e.target.value)} required />
                <input placeholder="Course" value={course}
                    onChange={e => setCourse(e.target.value)} required />
                <button type="submit">Add User</button>
            </form>
            <h2>Users from Backend</h2>
            {users.map(user =>
                <p key={user.id}>{user.name} - {user.course}</p>
            )}
        </div>
    );
}
export default App;
```

Run and verify

1. Terminal 1: cd $HOME\Downloads\MERN_Practicals\FullStackDemo
2. node server.js
3. Terminal 2: cd $HOME\Downloads\MERN_Practicals\SumDemo
4. npm run dev
5. Open Vite URL, add a name and course, and confirm the new user appears.

Important

> This compact demo uses an in-memory array. The journal Practical 10 uses MongoDB and includes full CRUD and validation; use the journal version if your examiner requires database persistence.

Terminal commands — quick reference

Purpose	Command
Go to Downloads	`cd $HOME\Downloads`
Go to main folder	`cd $HOME\Downloads\MERN_Practicals`
Create folder	`mkdir FolderName`
Enter folder	`cd FolderName`
Go one level up	`cd ..`
Initialize Node project	`npm init -y`
Install Express	`npm install express`
Install CORS	`npm install cors`
Install MongoDB driver	`npm install mongodb`
Create React project	`npm create vite@latest ProjectName -- --template react`
Install project dependencies	`npm install`
Run React app	`npm run dev`
Run Node file	`node filename.js`
Stop server	`Ctrl+C`

Common errors and fixes

- **Cannot find module express:** run npm install express inside the folder containing that server file.
- **Cannot find module mongodb:** run npm install mongodb inside that backend project.
- **Port already in use:** stop the other server with Ctrl+C or change PORT and use the matching URL.
- **Localhost does not open:** confirm the server terminal still shows it is running.
- **React import error:** check spelling, file extension and relative import path.
- **MongoDB connection error:** confirm MongoDB Community Server is installed and running before starting the API.
- **PowerShell path error:** use the full cd path in the practical and check folder names.

Journal alignment notes

This guide follows the question-bank order and uses the uploaded journal as the primary reference. Journal code is retained where the match is clear. For items without an exact standalone journal program, a small runnable implementation is supplied and identified as an adaptation. Practical 7 retains the journal CSS declarations but repairs the incomplete JSX wrapper so it compiles and applies the .container class. Practical 16 is a compact in-memory integration demonstration; the journal's full Practical 10 uses MongoDB and includes CRUD operations and validation. Follow your teacher's required version if it differs.